

11 Low Power Support

11.1 Power Saving Introduction

FreeRTOS offers an easy way to tap into low power modes with the IDLE task hooks and tickless Idle mode.

It is common to reduce the power consumed by the microcontroller on which FreeRTOS is running by using the IDLE task hook to place the microcontroller into a low power state. The power saving that can be achieved by this method is limited by the necessity to periodically exit and then re-enter the low power state to process tick interrupts. Further, if the frequency of the tick interrupt is too high (wake from idle is too frequent), the energy and time consumed entering and then exiting a low power state for every tick will outweigh any potential power saving gains for all but the lightest power saving modes.

FreeRTOS supports a low power state that allows the microcontroller to periodically enter and exit low power consumption. The FreeRTOS tickless idle mode stops the periodic tick interrupt during idle periods (when there are no application tasks that are able to execute), which allows the MCU to remain in a deep power saving state until either an interrupt occurs, or it is time for the RTOS kernel to transition a task into the Ready state. It then makes a correcting adjustment to the RTOS tick count value when the tick interrupt is restarted. The principle of the FreeRTOS tickless mode is to make the MCU enter the low-power mode to save system power consumption when the MCU is performing the idle task.

11.2 FreeRTOS Sleep Modes

There are three types of sleep modes supported in FreeRTOS:

1. `eAbortSleep` - This mode denotes a task has been made ready, a context switch was pended or a tick interrupt has already occurred but was pended since the scheduler was suspended. It signals the RTOS to abort entering a sleep mode.
2. `eStandardSleep` - This mode allows to enter a sleep mode that will not last longer than the expected idle time.
3. `eNoTasksWaitingTimeout` - This mode is entered when no tasks are waiting for a timeout so it is safe to enter a sleep mode that can only be exited by an external interrupt or reset.

11.3 Functions and Enabling Built-in Tickless Idle Functionality

The Built-in Tickless Idle functionality is enabled by defining `configUSE_TICKLESS_IDLE` as 1 in `FreeRTOSConfig.h` (for ports that support this feature). User defined tickless idle functionality can be provided for any FreeRTOS port (including those that include a built in implementation) by defining `configUSE_TICKLESS_IDLE` to 2 in `FreeRTOSConfig.h`.

When the tickless idle functionality is enabled, the kernel will call the `portSUPPRESS_TICKS_AND_SLEEP()` macro when the following two conditions are satisfied:

1. The Idle task is the only task able to run because all the application tasks are either in the Blocked state or in the Suspended state.

2. At least n further complete tick periods will pass before the kernel is due to transition an application task out of the Blocked state, where n is set by the `configEXPECTED_IDLE_TIME_BEFORE_SLEEP` definition in `FreeRTOSConfig.h`.

11.3.1 The `portSUPPRESS_TICKS_AND_SLEEP()` Macro

```
portSUPPRESS_TICKS_AND_SLEEP( xExpectedIdleTime )
```

Listing 11.1 The prototype for the `portSUPPRESS_TICKS_AND_SLEEP` macro

The value of the `xExpectedIdleTime` parameter in `portSUPPRESS_TICKS_AND_SLEEP()` equals the total number of tick periods before a task is due to be moved into the Ready state. The parameter value is therefore the time the microcontroller can safely remain in a deep sleep state, with the tick interrupt suppressed.

11.3.2 The `vPortSuppressTicksAndSleep` Function

The `vPortSuppressTicksAndSleep()` function is defined in FreeRTOS and it can be used to implement the tickless mode. This function is weakly defined in the FreeRTOS Cortex-M port layer and can be overridden by the application writer.

```
void vPortSuppressTicksAndSleep( TickType_t xExpectedIdleTime );
```

Listing 11.2 The `vPortSuppressTicksAndSleep` API function prototype

11.3.3 The `eTaskConfirmSleepModeStatus` Function

The API `eTaskConfirmSleepModeStatus` returns the sleep mode status to determine if it is ok to proceed with the sleep and if it is ok to sleep indefinitely. This functionality is only available when `configUSE_TICKLESS_IDLE` is set to 1.

```
eSleepModeStatus eTaskConfirmSleepModeStatus( void );
```

Listing 11.3 The `eTaskConfirmSleepModeStatus` API function prototype

If `eTaskConfirmSleepModeStatus()` returns `eNoTasksWaitingTimeout` when it is called from within `portSUPPRESS_TICKS_AND_SLEEP()`, then the microcontroller can remain in a deep sleep state indefinitely. `eTaskConfirmSleepModeStatus()` will only return `eNoTasksWaitingTimeout` when the following conditions are true:

- Software timers are not being used, so the scheduler is not due to execute a timer callback function at any time in the future.
- All the application tasks are either in the Suspended state, or in the Blocked state with a timeout value of `portMAX_DELAY`, so the scheduler is not due to transition a task out of the Blocked state at any fixed time in the future.

To avoid race conditions, the FreeRTOS scheduler is suspended before `portSUPPRESS_TICKS_AND_SLEEP()` is called, and resumed when `portSUPPRESS_TICKS_AND_SLEEP()` completes. This ensures application tasks cannot execute between the microcontroller exiting its low power state and `portSUPPRESS_TICKS_AND_SLEEP()` completing its execution. Further, it is necessary for the `portSUPPRESS_TICKS_AND_SLEEP()` function to create a small critical section between the timer being stopped and the sleep mode being entered to ensure it is ok to proceed into the sleep mode. `eTaskConfirmSleepModeStatus()` should be called from this critical section.

In addition, FreeRTOS provides users with two other interface functions defined in `FreeRTOSConfig.h`. These macros allow the application writer to add additional steps before and after the MCU is placed into the low power state, respectively.

11.3.4 The `configPRE_SLEEP_PROCESSING` configuration

```
configPRE_SLEEP_PROCESSING( xExpectedIdleTime )
```

Listing 11.4 *The prototype for the `configPRE_SLEEP_PROCESSING` macro*

Before the user can make the MCU enter the low-power mode, `configPRE_SLEEP_PROCESSING()` must be called to configure the system parameters to reduce the system power consumption, such as turning off other peripheral clocks, reducing the system frequency.

11.3.5 The `configPOST_SLEEP_PROCESSING` configuration

```
configPOST_SLEEP_PROCESSING( xExpectedIdleTime )
```

Listing 11.5 *The prototype for the `configPOST_SLEEP_PROCESSING` macro*

After exiting the low-power mode, the user should call the `configPOST_SLEEP_PROCESSING()` function to restore the system's main frequency and peripheral functions.

11.4 Implementing `portSUPPRESS_TICKS_AND_SLEEP()` Macro

If the FreeRTOS port in use does not provide a default implementation of `portSUPPRESS_TICKS_AND_SLEEP()`, then the application writer can provide their own implementation by defining `portSUPPRESS_TICKS_AND_SLEEP()` in `FreeRTOSConfig.h`. If the FreeRTOS port in use does provide a default implementation of `portSUPPRESS_TICKS_AND_SLEEP()`, then the application writer can override the default implementation by defining `portSUPPRESS_TICKS_AND_SLEEP()` in `FreeRTOSConfig.h`.

The following source code is an example of how `portSUPPRESS_TICKS_AND_SLEEP()` might be implemented by an application writer. The example is basic, and will introduce some slippage between the time maintained by the kernel and calendar time. Of the function calls shown in the example, only `vTaskStepTick()` and `eTaskConfirmSleepModeStatus()` are part of the FreeRTOS API. The other functions are specific to the clocks and power saving modes available on the hardware in use, and as such, must be provided by the application writer.

```

/* First define the portSUPPRESS_TICKS_AND_SLEEP() macro. The parameter is the
   time, in ticks, until the kernel next needs to execute. */

#define portSUPPRESS_TICKS_AND_SLEEP( xIdleTime ) vApplicationSleep( xIdleTime )

/* Define the function that is called by portSUPPRESS_TICKS_AND_SLEEP(). */
void vApplicationSleep( TickType_t xExpectedIdleTime )
{
    unsigned long ulLowPowerTimeBeforeSleep, ulLowPowerTimeAfterSleep;

    eSleepModeStatus eSleepStatus;

    /* Read the current time from a time source that will remain operational
       while the microcontroller is in a low power state. */
    ulLowPowerTimeBeforeSleep = ulGetExternalTime();

    /* Stop the timer that is generating the tick interrupt. */
    prvStopTickInterruptTimer();

    /* Enter a critical section that will not effect interrupts bringing the MCU
       out of sleep mode. */
    disable_interrupts();

    /* Ensure it is still ok to enter the sleep mode. */
    eSleepStatus = eTaskConfirmSleepModeStatus();

    if( eSleepStatus == eAbortSleep )
    {
        /* A task has been moved out of the Blocked state since this macro was
           executed, or a context switch is being held pending. Do not enter a
           sleep state. Restart the tick and exit the critical section. */
        prvStartTickInterruptTimer();
        enable_interrupts();
    }
    else
    {
        if( eSleepStatus == eNoTasksWaitingTimeout )
        {
            /* It is not necessary to configure an interrupt to bring the
               microcontroller out of its low power state at a fixed time in
               the future. */
            prvSleep();
        }
        else
        {
            /* Configure an interrupt to bring the microcontroller out of its low
               power state at the time the kernel next needs to execute. The
               interrupt must be generated from a source that remains operational
               when the microcontroller is in a low power state. */
            vSetWakeTimeInterrupt( xExpectedIdleTime );

            /* Enter the low power state. */
            prvSleep();
        }
    }
}

```

```

will
    /* Determine how long the microcontroller was actually in a low power
       state for, which will be less than xExpectedIdleTime if the
       microcontroller was brought out of low power mode by an interrupt
       other than that configured by the vSetWakeTimeInterrupt() call.
       Note that the scheduler is suspended before
       portSUPPRESS_TICKS_AND_SLEEP() is called, and resumed when
       portSUPPRESS_TICKS_AND_SLEEP() returns. Therefore no other tasks
       execute until this function completes. */
    ulLowPowerTimeAfterSleep = ulGetExternalTime();

    /* Correct the kernels tick count to account for the time the
       microcontroller spent in its low power state. */
    vTaskStepTick( ulLowPowerTimeAfterSleep - ulLowPowerTimeBeforeSleep );
}

/* Exit the critical section - it might be possible to do this immediately
   after the prvSleep() calls. */
enable_interrupts();

/* Restart the timer that is generating the tick interrupt. */
prvStartTickInterruptTimer();
}
}

```

Listing 11.6 An example of a user defined implementation of `portSUPPRESS_TICKS_AND_SLEEP()`

11.5 Idle Task Hook Function

The Idle task can optionally call an application defined hook (or callback) function - the idle hook. The idle task runs at the lowest priority, so such an idle hook function will only get executed when there are no tasks of higher priority that are able to run. This makes the Idle hook function an ideal place to put the processor into a low power state - providing an automatic power saving whenever there is no processing to be performed. The Idle hook will only get called if `configUSE_IDLE_HOOK` is set to 1 within `FreeRTOSConfig.h`.

```
void vApplicationIdleHook( void );
```

Listing 11.7 The `vApplicationIdleHook` API function prototype

The idle hook is called repeatedly as long as the idle task is running. It is paramount that the idle hook function does not call any API functions that could cause it to block. Also, if the application makes use of the `vTaskDelete()` API function then the idle task hook must be allowed to periodically return, since the idle task is responsible for cleaning up the resources that were allocated by the RTOS kernel to the task that has been deleted.